Expérience de l'article *DALL-E-Bot: Introducing Web-Scale Diffusion Models to Robotics*

1. Objectif général de l'expérience

L'article étudie une question précise : **un modèle de génération d'images entraîné à très grande échelle peut-il servir à définir automatiquement un état final désirable pour un robot ?**

Le problème choisi est le **réarrangement d'objets**. Au départ, plusieurs objets sont posés de manière désordonnée sur une table. Le système doit déterminer une disposition finale naturelle, puis commander un robot pour déplacer les objets vers cette disposition.

L'idée centrale de DALL-E-Bot est d'utiliser DALL-E 2 comme un **« moteur d'imagination »** (*imagination engine*). DALL-E ne commande pas directement le robot. Il produit une image représentant une organisation plausible des objets. Les autres modules du système transforment ensuite cette image imaginée en positions physiques que le robot peut atteindre.

Le fonctionnement global peut être résumé ainsi :

1. observer la scène réelle avec une caméra ;
2. détecter et décrire chaque objet ;
3. construire un *prompt* textuel ;
4. demander à DALL-E de générer plusieurs arrangements possibles ;
5. sélectionner l'image générée la plus compatible avec les objets réels ;
6. associer chaque objet réel à son équivalent dans l'image générée ;
7. estimer la position et l'orientation finales de chaque objet ;
8. convertir ces résultats en coordonnées physiques ;
9. exécuter les déplacements avec le robot ;
10. évaluer la qualité des arrangements obtenus.

Le système est qualifié de **zero-shot** : les auteurs n'entraînent pas un nouveau modèle sur des exemples propres aux scènes de l'expérience. Ils utilisent des modèles déjà pré-entraînés, notamment DALL-E 2, Mask R-CNN, OFA et CLIP. Il n'y a donc pas de collecte d'un jeu d'exemples montrant au robot comment dresser une table, ranger un bureau ou placer des fruits avant d'effectuer ces tâches.

1.1 Statut réel de la reproductibilité

L'article décrit précisément l'architecture logique, mais **pas tous les paramètres nécessaires à une reproduction bit à bit**. Aucun dépôt officiel de code n'est indiqué dans l'article ou sur la page du projet. Il faut donc distinguer deux catégories dans la suite de ce document :

- **Spécifié par l'article** : composant, modèle ou opération explicitement donné par les auteurs ;
- **Choix de reproduction** : décision d'ingénierie nécessaire lorsque l'article ne donne pas la valeur exacte.

Les informations suivantes sont spécifiées :

- Mask R-CNN de Detectron2, pré-entraîné sur LVIS ;
- OFA pour le *captioning* image-vers-texte ;

- un modèle visuel CLIP produisant un vecteur de 512 dimensions ;
- Flair pour le *Part-of-Speech tagging* ;
- DALL-E 2 par l'API OpenAI, avec quatre images par lot ;
- similarité cosinus et algorithme hongrois pour l'appariement ;
- ICP sur les pixels des masques pour obtenir (x, y, θ) ;
- caméra de profondeur montée au poignet ;
- robot Franka Emika Panda et ventouse souple ;
- exécution par *pick-and-place*.

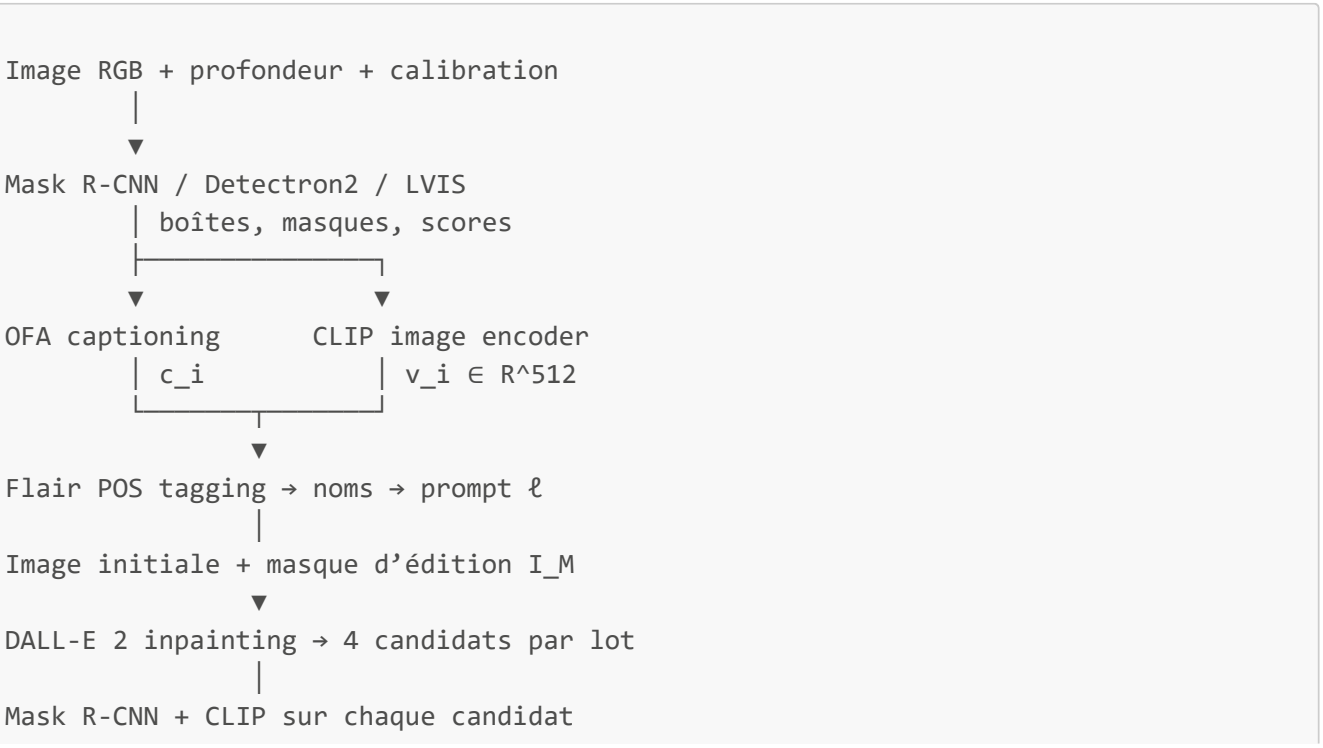
En revanche, l'article ne précise pas :

- le fichier de configuration et le checkpoint LVIS exacts de Detectron2 ;
- le seuil de confiance, le seuil NMS et le traitement des détections qui se chevauchent ;
- la variante ou le checkpoint exact d'OFA ;
- l'architecture exacte de CLIP, malgré la dimension 512 annoncée ;
- le checkpoint Flair exact et la règle complète d'extraction en cas de plusieurs noms ;
- le nombre de pixels utilisé pour agrandir les masques et effacer les ombres ;
- la largeur de la bande représentant les bords de la table ;
- les paramètres exacts d'ICP, son critère d'arrêt et son initialisation ;
- les seuils de collision et les coefficients de correction d'échelle ;
- le modèle de caméra de profondeur, les matrices de calibration et le contrôleur du robot ;
- les vitesses, hauteurs de sécurité, seuils de préhension et détails complets du planificateur.

Une reproduction scientifique honnête doit donc enregistrer tous ces choix dans un fichier de configuration et les rapporter avec les résultats. Toute valeur proposée ci-dessous lorsqu'elle n'est pas dans l'article est clairement marquée **choix de reproduction**.

1.2 Architecture logicielle à reconstruire

Une implémentation doit séparer les modules suivants :



```
                ▼
Filtre sur le nombre d'objets
                ▼
Similarités cosinus + Hungarian matching
                ▼
Meilleure image objectif I_G
                ▼
Redimensionnement des masques + ICP
                ▼
Correction d'échelle + résolution des collisions
                ▼
Projection pixel → table avec profondeur/calibration
                ▼
Planification pick-and-place → Franka Panda
```

Il est recommandé de conserver un journal pour chaque exécution : image RGB, carte de profondeur, détections, masques, légendes OFA, noms retenus, vecteurs CLIP, *prompt*, masque envoyé au générateur, quatre images candidates, matrices de similarité, affectation hongroise, transformations ICP et poses robotiques. Sans ces données intermédiaires, il est presque impossible de localiser l'origine d'un échec.

1.3 Protocole reproductible, étape par étape

Étape R0 — Fixer les conventions et enregistrer la configuration

But. Éviter qu'une ambiguïté de repère ou de masque produise un résultat apparemment plausible mais faux.

À définir avant toute expérience :

- résolution commune des images ;
- convention des pixels : origine en haut à gauche, u vers la droite, v vers le bas ;
- convention de l'angle θ, avec sens positif explicitement choisi ;
- repère caméra, repère robot et repère table ;
- liste des objets mobiles et immobiles ;
- seuils de détection et de collision ;
- graine aléatoire de tous les modules locaux ;
- versions de Python, PyTorch, Detectron2, OFA, CLIP, Flair, OpenCV/SciPy et de l'API d'image.

Sortie à sauvegarder. Un fichier de configuration daté. Les réponses d'un service génératif distant peuvent changer même avec le même *prompt* ; il faut donc aussi sauvegarder toutes les images reçues et leurs métadonnées.

Étape R1 — Calibrer la caméra et définir le plan de la table

Technique utilisée. Calibration intrinsèque et extrinsèque d'une caméra RGB-D montée sur le poignet.

Les paramètres intrinsèques peuvent être représentés par :

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Pour un pixel (u, v) de profondeur z, le point dans le repère caméra est :

$$X_c = \frac{(u - c_x)z}{f_x}, \qquad Y_c = \frac{(v - c_y)z}{f_y}, \qquad Z_c = z.$$

Une transformation rigide ${}^{B}T_{C}$, obtenue par calibration *hand-eye*, convertit ensuite le point du repère caméra C vers le repère de base du robot B. Le plan de table doit être estimé ou mesuré pour projeter les cibles à une hauteur sûre.

Spécifié par l'article. Caméra de profondeur au poignet et projection des poses dans l'espace 3D de la table.

Non spécifié. Modèle de caméra, procédure de calibration, filtrage de profondeur et précision attendue.

Choix de reproduction conseillé. Utiliser une mire, sauvegarder K, les coefficients de distorsion et ${}^{B}T_{C}$, puis vérifier l'erreur de reprojection sur plusieurs points de la table avant toute manipulation.

Étape R2 — Capturer et normaliser l'observation initiale

Entrées. Image RGB I_I, profondeur alignée D_I, calibration.

Traitement. Capturer une vue aussi proche que possible d'une vue verticale. Corriger la distorsion optique. Si le générateur impose une image carrée, recadrer ou redimensionner RGB, profondeur et masques avec exactement la même transformation.

Pourquoi. Les coordonnées de l'image générée seront interprétées comme des positions sur la table. Un recadrage appliqué seulement au RGB et non à la profondeur produirait des positions 3D erronées.

Sorties. Image RGB normalisée, profondeur alignée et transformation permettant de revenir aux pixels originaux.

Étape R3 — Détecter et segmenter les objets

Modèle. Mask R-CNN dans Detectron2, pré-entraîné sur LVIS, environ 1 200 classes.

Entrée. I_I.

Procédure :

1. exécuter l'inférence Detectron2 sans réentraînement ;
2. récupérer pour chaque instance le score, la boîte B_i, le masque M_i et la classe LVIS ;
3. supprimer les détections sous le seuil choisi ;
4. vérifier que chaque objet physique utile correspond à une seule instance ;
5. convertir les masques probabilistes en masques binaires ;
6. sauvegarder les masques dans la résolution exacte de I_I.

Usage des sorties. La boîte sert à créer le *crop*. Le masque sert plus tard à effacer l'objet pour l'*inpainting*, à calculer les collisions et à estimer sa pose par ICP. La classe Mask R-CNN n'est pas utilisée dans la méthode complète pour créer le *prompt*.

Non spécifié. Checkpoint Detectron2 exact et seuils.

Choix de reproduction. Tester un modèle LVIS Mask R-CNN du *Model Zoo* de Detectron2, documenter son nom exact, commencer avec un seuil de score fixé dans la configuration, puis ne plus le modifier pendant l'évaluation. Cette valeur est une décision de reproduction, pas une valeur donnée dans l'article.

Étape R4 — Découper chaque objet

Technique. *Cropping* à partir de la boîte englobante.

Pour chaque objet, extraire :

$$O_i = I_I[B_i].$$

Le papier parle de *RGB crops around each object's bounding box*. Il faut donc conserver un peu de contexte autour de l'objet si l'implémentation le prévoit, mais la marge exacte n'est pas fournie.

Choix de reproduction. Choisir une marge fixe, en pixels ou en pourcentage de la boîte, la consigner, couper la boîte aux limites de l'image, puis utiliser exactement le même protocole pour les objets des images générées.

Attention. Un fond trop large peut influencer CLIP et OFA ; un recadrage trop serré peut couper le manche d'un couvert. La marge doit être validée sur un ensemble de développement distinct des essais finaux.

Étape R5 — Générer la légende c_i

Modèle. OFA (*One-For-All*) en mode *image-to-text captioning*.

Entrée. Crop RGB O_i.

Sortie. Une phrase c_i, par exemple *"a red apple on a wooden table"*.

Procédure :

1. appliquer le prétraitement exigé par le checkpoint OFA : redimensionnement, normalisation et tokenisation de la requête ;
2. demander une légende de l'image ;
3. décoder la séquence de texte ;
4. normaliser espaces et ponctuation ;
5. conserver la sortie brute pour l'audit.

Spécifié. Le modèle OFA et l'utilisation séparée de chaque crop.

Non spécifié. Checkpoint OFA, taille d'entrée, *beam size*, longueur maximale et texte de la requête.

Reproduction. Le checkpoint et les paramètres de décodage doivent être gelés avant l'évaluation. Une inspection humaine peut servir au débogage, mais corriger manuellement les légendes pendant les essais détruirait le caractère autonome de l'expérience.

Étape R6 — Extraire le vecteur CLIP v_i

Modèle. Encodeur visuel CLIP pré-entraîné ; la sortie annoncée est de dimension 512.

Entrée. Le même crop O_i.

Procédure :

1. redimensionner et normaliser le crop avec le prétraitement officiel du checkpoint CLIP choisi ;
2. calculer $z_i = \mathrm{CLIPImage}(O_i)$;
3. normaliser le vecteur :

$$v_i = \frac{z_i}{\|z_i\|_2}.$$

Avec cette normalisation, le produit scalaire $v_i^T v_j$ est égal à la similarité cosinus.

Usage. Le vecteur ne sert pas à créer le *prompt*. Il sert à associer les objets réels aux objets générés et à noter les images candidates.

Non spécifié. Backbone CLIP exact. Plusieurs architectures peuvent produire 512 dimensions ; on ne peut donc pas le déduire de manière certaine.

Choix de reproduction. Choisir un checkpoint CLIP public précis, noter son identifiant et l'utiliser sans changement pour toutes les images. Changer de backbone modifie l'expérience.

Étape R7 — Extraire les noms et construire le *prompt*

Modèle. Modèle d'étiquetage morpho-syntaxique de Flair.

Entrée. Ensemble des légendes $\{c_i\}$.

Traitement :

1. tokeniser chaque légende ;
2. appliquer le *POS tagger* Flair ;
3. identifier le nom qui désigne l'objet principal ;
4. convertir si nécessaire au singulier ;
5. conserver les occurrences multiples : deux pommes doivent rester deux éléments dans la liste ;
6. construire une énumération en anglais ;
7. ajouter la chaîne *"top-down"*.

Exemple :

```
c1 = "a fork with a black handle on a wooden table" → fork
c2 = "an empty white plate on a wooden table"       → plate
prompt = "A fork, a plate, top-down"
```

Non spécifié. Règle lorsque la légende contient plusieurs noms, par exemple *"a cup next to a plate"*.

Choix de reproduction. Définir une règle déterministe, par exemple sélectionner le premier groupe nominal dont la tête n'est pas un mot de fond tel que *table*. Journaliser le résultat et compter comme erreur toute mauvaise extraction ; ne pas intervenir manuellement pendant un essai.

Étape R8 — Construire le masque d'édition I_M

Le terme employé dans l'article est **image mask** ou masque d'*inpainting*, pas « masque d'erreur ». Son rôle est de dire au générateur quels pixels peuvent être reconstruits.

Définissons :

- $S_{mov} = \bigvee_{i \in \mathcal{M}} M_i$, union des masques des objets mobiles ;
- $S_{fix} = \bigvee_{i \in \mathcal{F}} M_i$, union des masques des objets fixes ;
- S_{edge}, bande correspondant aux bords visibles de la table ;
- $\mathrm{dilate}(S_{mov}, r)$, dilatation morphologique de rayon r.

La région modifiable conceptuelle est :

$$E = \mathrm{dilate}(S_{mov}, r)$$

tandis que les régions à préserver comprennent au minimum S_{fix} et S_{edge}. L'image d'entrée à éditer doit montrer les objets fixes et les bords, mais les objets mobiles et leurs ombres doivent se trouver dans la région modifiable.

Pourquoi dilater ? Le masque de segmentation couvre l'objet mais pas toujours son ombre. Sans dilatation, des pixels résiduels peuvent pousser DALL-E à reconstruire l'objet au même endroit.

Trois usages du masque selon l'article :

1. préserver les objets que le robot n'a pas le droit de déplacer ;
2. préserver des bandes sur les bords de la table pour l'ancrage visuel ;
3. rendre éditables les objets mobiles, avec des masques agrandis pour supprimer leurs ombres.

Point critique d'implémentation. Les API n'utilisent pas toutes la même convention de masque. Certaines interprètent l'alpha transparent comme « à modifier » et l'opaque comme « à conserver » ; d'autres attendent un masque binaire inverse. Il faut tester cette convention avec un exemple minimal avant l'expérience. Une inversion silencieuse du masque détruirait le protocole.

Non spécifié. Rayon r, forme du noyau morphologique, largeur de S_{edge}, méthode de détection de la table et convention binaire exacte utilisée dans le script des auteurs.

Choix de reproduction. Définir ces valeurs dans la configuration, sauvegarder une visualisation colorée `conserver/modifier`, et mesurer le pourcentage de l'image modifiable. Pour une reproduction historique stricte, utiliser le comportement de l'API DALL-E 2 de la période de l'article. Employer un modèle moderne constitue une adaptation, pas une reproduction exacte.

Étape R9 — Générer les images candidates

Modèle. DALL-E 2 via l'API OpenAI, en mode édition/*inpainting*.

Entrées. Image initiale préparée, masque I_M, *prompt* ℓ.

Sortie. Un lot de quatre images candidates $\{I_G^k\}_{k=1}^4$.

Boucle :

```
répéter :
    demander 4 éditions avec la même image, le même masque et le même prompt
    sauvegarder immédiatement les 4 réponses et métadonnées
    analyser les 4 images
jusqu'à ce qu'au moins une image contienne le bon nombre d'objets
```

Non spécifié. Résolution exacte de la requête, éventuels paramètres internes du service et nombre maximal de nouveaux lots.

Reproduction. Ajouter une limite d'appels pour éviter une boucle infinie et déclarer un échec lorsque la limite est atteinte. Le nombre de lots nécessaires est lui-même une métrique de robustesse utile.

Étape R10 — Segmenter les objets générés

Modèles. Même Mask R-CNN/Detectron2 et même CLIP qu'aux étapes R3 et R6.

Pour chaque candidat I_G^k :

1. exécuter Mask R-CNN avec les mêmes seuils ;
2. obtenir les masques $M_{G,j}^k$;
3. produire les crops avec la même marge ;
4. calculer les vecteurs CLIP normalisés $v_{G,j}^k$.

OFA n'est pas nécessaire pour l'appariement à cette étape : la similarité est fondée sur CLIP. Utiliser exactement le même pipeline des deux côtés évite un décalage de représentation.

Filtre. Rejeter le candidat si le nombre d'instances détectées n'est pas égal à n, le nombre d'objets de la scène initiale.

Étape R11 — Apparier les objets et choisir l'image

Pour chaque candidat conservé, construire la matrice :

$$S_{ij}^k = v_{I,i}^T v_{G,j}^k.$$

Appliquer l'algorithme hongrois au coût $-S^k$, puisque les implémentations standard minimisent souvent un coût alors que l'article veut maximiser la similarité.

Si $\pi_k(i)$ est l'objet généré affecté à l'objet réel i, le score du candidat est :

$$Q_k = \sum_{i=1}^{n} S_{i,\pi_k(i)}^k.$$

Sélectionner :

$$k^* = \arg\max_k Q_k.$$

À sauvegarder. Matrice S^k, permutation π_k, score Q_k et montage visuel des paires. Cela permet de voir si l'échec vient de DALL-E ou de CLIP.

Non spécifié. Seuil minimal de similarité. L'article choisit le meilleur score parmi les candidats de cardinalité correcte.

Étape R12 — Estimer la transformation 2D avec ICP

Pour chaque paire appariée $(M_{I,i}, M_{G,\pi(i)})$:

1. calculer les boîtes englobantes des deux masques ;
2. redimensionner le masque initial pour que sa boîte ait les dimensions de la boîte générée ;
3. convertir les pixels de premier plan des deux masques en deux nuages de points 2D ;
4. centrer initialement les nuages sur leurs centroïdes ;
5. appliquer ICP rigide 2D pour estimer rotation $R(\theta)$ et translation $t = (x, y)$;
6. recomposer la transformation en tenant compte du redimensionnement et des coordonnées des boîtes dans l'image complète ;
7. produire la pose cible du centre de l'objet et son angle cible.

La minimisation ICP peut s'écrire :

$$\min_{R,t} \sum_{p \in P_I} \|Rp + t - q(p)\|^2,$$

où $q(p)$ est le point correspondant le plus proche dans le masque cible.

Attention aux symétries. Une forme presque symétrique peut produire plusieurs minima, notamment une erreur de 180°. L'article final reconnaît cette limitation mais ne fournit pas un correctif complet reproductible.

Non spécifié. Sous-échantillonnage des pixels, métrique robuste, nombre d'itérations, tolérance, initialisations multiples et règle de sélection.

Choix de reproduction. Déclarer ces paramètres, utiliser plusieurs initialisations angulaires si nécessaire, conserver l'erreur finale d'alignement et rejeter ou signaler une transformation au-dessus d'un seuil fixé avant le test.

Étape R13 — Corriger l'échelle et les collisions

Problème. Les distances dans l'image DALL-E ne sont pas nécessairement à l'échelle du monde réel.

Procédure décrite :

1. trouver l'objet central, celui dont la somme des distances aux autres est minimale ;
2. comparer les tailles apparentes entre scène réelle et image générée ;
3. rapprocher ou éloigner les autres poses par rapport à l'objet central ;
4. tester les intersections entre les masques transformés ;
5. en cas de collision, éloigner progressivement les objets du centre jusqu'à disparition des intersections.

Technique. Correction géométrique radiale suivie d'une résolution de collisions dans l'espace image.

Non spécifié. Formule exacte du facteur d'échelle, pas de déplacement et marge minimale entre objets.

Reproduction. Écrire explicitement la formule choisie, par exemple à partir d'un rapport robuste des tailles de boîtes, puis utiliser une marge de sécurité en pixels convertie en centimètres. Les valeurs doivent être fixées avant les essais finaux.

Étape R14 — Convertir la pose image en pose robot

Pour chaque centre cible (u_i^*, v_i^*), récupérer une profondeur fiable. Si la région cible est vide dans l'image initiale, la profondeur brute à ce pixel représente normalement la table ; on peut utiliser le plan de table calibré plutôt qu'une mesure bruitée.

Projeter ensuite le point vers le repère caméra avec K^{-1}, puis vers le repère robot avec ${}^{B}T_{C}$. L'angle dans l'image doit être converti en rotation autour de la normale à la table, en tenant compte de l'orientation de la caméra.

Contrôles indispensables :

- la cible est dans l'espace atteignable ;
- l'objet ne dépasse pas de la table ;
- aucune collision finale n'est prévue ;
- la ventouse dispose d'une zone de contact ;
- l'orientation du poignet respecte ses limites articulaires.

Non spécifié. Méthode exacte de choix du point de préhension et contrôles cinématiques.

Étape R15 — Planifier et exécuter les *pick-and-place*

Matériel. Franka Emika Panda et *compliant suction gripper*.

Pour chaque objet :

1. calculer un point de saisie, généralement à l'intérieur du masque et loin du contour ;
2. déplacer l'effecteur vers une pose d'approche au-dessus de l'objet ;
3. descendre verticalement ;
4. activer la ventouse et vérifier la préhension ;
5. remonter à une hauteur de transport ;
6. se déplacer au-dessus de la cible ;
7. appliquer l'orientation cible ;
8. descendre, relâcher l'objet et remonter ;
9. capturer une image de vérification.

Planificateur de conflits. Si un objet bloque la destination d'un autre, le placer temporairement dans une zone tampon, déplacer l'objet concerné, puis reprendre l'objet temporaire et le placer à sa cible.

Non spécifié. Algorithme complet d'ordonnancement, trajectoires, vitesses et vérification de préhension.

Reproduction sûre. Commencer en simulation ou avec des mouvements à vitesse limitée, définir des hauteurs d'approche et de transport, surveiller la force et prévoir un arrêt d'urgence. Ces mesures sont nécessaires à une mise en œuvre réelle mais ne sont pas des paramètres fournis par l'article.

Étape R16 — Vérifier le résultat et calculer les métriques

Après exécution, reprendre une image RGB depuis la même vue.

Pour l'expérience de réarrangement complet, reproduire le protocole utilisateur : mêmes images pour tous les participants, ordre contrôlé ou randomisé, question et échelle de Likert identiques, cinq initialisations par

méthode et par scène.

Pour l'expérience d'objet manquant :

1. conserver les poses acceptables données par chaque participant ;
2. aligner le masque prédit sur chaque masque acceptable ;
3. calculer l'erreur de translation en centimètres et l'erreur d'angle en degrés ;
4. garder l'erreur par rapport à la pose acceptable la plus proche ;
5. rapporter la médiane entre participants.

Il faut aussi rapporter des métriques de pipeline qui ne figurent pas nécessairement dans le tableau principal : taux d'échec de détection, proportion de légendes incorrectes, nombre moyen de lots DALL-E, taux d'images rejetées, taux d'appariements incorrects, erreur de calibration et taux de réussite de la préhension.

1.4 Pseudo-code complet de la méthode

```
ENTRÉES
    image initiale RGB I_I
    profondeur D_I
    calibration K et transformation caméra→robot
    ensemble des objets immobiles F

1. detections_I ← MaskRCNN_LVIS(I_I)
2. pour chaque détection i :
       M_I[i] ← masque binaire
       O_I[i] ← crop autour de la bounding box
       c[i]   ← OFA_caption(O_I[i])
       v_I[i] ← normalize(CLIP_image(O_I[i]))

3. noms ← Flair_POS_extract_object_nouns(c)
4. ℓ ← format_enumeration(noms) + ", top-down"

5. S_mov  ← union des M_I[i] mobiles
6. S_fix  ← union des M_I[i] immobiles
7. S_edge ← masque des bords de table
8. I_M ← convertir(dilate(S_mov), S_fix, S_edge)
          selon la convention du masque de l'API

9. candidats_valides ← ∅
10. tant que candidats_valides est vide et budget non dépassé :
        candidats ← DALL-E2_edit(I_I, I_M, ℓ, n=4)
        pour chaque image k :
            detections_G[k] ← MaskRCNN_LVIS(I_G[k])
            si nombre(detections_G[k]) = nombre(detections_I) :
                calculer M_G[k] et v_G[k]
                ajouter k à candidats_valides

11. pour chaque candidat valide k :
        S_k[i,j] ← cosine(v_I[i], v_G[k][j])
        π_k ← Hungarian(cost = -S_k)
        Q_k ← somme_i S_k[i, π_k(i)]
```

```
12. k* ← argmax_k Q_k

13. pour chaque objet réel i :
        masque_source ← M_I[i]
        masque_cible  ← M_G[k*][π_k*(i)]
        redimensionner masque_source selon la boîte cible
        T_2D[i] ← ICP(masque_source, masque_cible)

14. poses_2D ← correction_echelle_et_collisions(T_2D)
15. poses_3D ← projection_RGBD_et_calibration(poses_2D)
16. plan ← ordonnancer_pick_and_place_avec_zones_tampons(poses_3D)
17. exécuter(plan)
18. capturer et évaluer le résultat
```

1.5 Tests unitaires nécessaires avant l'expérience complète

Pour rendre la reproduction diagnostiquable, chaque module doit être testé séparément :

1. **Segmentation** : afficher chaque M_i sur l'image et vérifier le nombre d'objets.
2. **OFA** : constituer une petite table `crop → légende brute → nom extrait`.
3. **CLIP** : vérifier que les paires du même type d'objet ont une similarité supérieure aux paires différentes.
4. **Masque d'édition** : envoyer une image simple et confirmer visuellement quelles zones sont modifiées.
5. **Hungarian matching** : tester une matrice de similarité artificielle dont l'affectation optimale est connue.
6. **ICP** : appliquer à un masque une transformation connue, puis vérifier que l'algorithme retrouve (x, y, θ).
7. **Projection 3D** : sélectionner des pixels correspondant à des marques mesurées sur la table et calculer l'erreur en millimètres.
8. **Collision** : vérifier que deux masques transformés ne se recouvrent pas avec la marge choisie.
9. **Robot** : tester la saisie et le dépôt d'un seul objet vers une pose imposée avant d'utiliser une pose issue de DALL-E.

Ce découpage est essentiel : si l'arrangement final échoue, il permet de savoir si la cause est sémantique, géométrique ou robotique.

---

2. Entrée du système : une observation RGB de la scène

Le système reçoit une image couleur initiale, notée :

$$I_I$$

où l'indice I signifie *initial*. Cette image est capturée depuis le dessus de la table par une caméra montée sur le poignet du robot.

Une image RGB contient des valeurs de couleur rouge, verte et bleue pour chaque pixel. Elle ne dit toutefois pas explicitement au robot : « voici une assiette », « voici sa forme » ou « voici sa position ». La première partie de la méthode consiste donc à convertir les pixels en une représentation structurée, objet par objet.

La vue du dessus, appelée **top-down view**, facilite la tâche :

- les déplacements principaux se font dans le plan de la table ;
- la position d'un objet peut être décrite principalement par deux coordonnées (x, y) ;
- son orientation peut être représentée par un angle θ autour de l'axe vertical ;
- les masques 2D des objets sont plus faciles à comparer entre l'image réelle et l'image générée.

---

3. Construction d'une représentation au niveau des objets

Pour chaque objet i, le système construit le triplet suivant :

$$(M_i, c_i, v_i)$$

Les trois éléments ont des fonctions différentes :

- M_i décrit **où se trouve l'objet et quelle est sa forme visible** ;
- c_i décrit **l'objet avec du texte** ;
- v_i décrit **son contenu visuel et sémantique sous forme numérique**.

Cette séparation est importante. Un seul type d'information ne suffirait pas à toutes les étapes : le texte sert à construire le *prompt*, les caractéristiques CLIP servent à reconnaître les correspondances, et le masque sert à calculer les poses.

3.1 Détection et segmentation avec Mask R-CNN

L'image initiale est d'abord traitée par **Mask R-CNN**, à travers la bibliothèque Detectron2. Le modèle utilisé a été pré-entraîné sur **LVIS**, un jeu de données comprenant environ 1 200 classes d'objets.

Pour chaque objet détecté, Mask R-CNN fournit :

- une **bounding box**, c'est-à-dire un rectangle qui entoure l'objet ;
- une classe supposée, par exemple *plate* ou *fork* ;
- un **segmentation mask** M_i, c'est-à-dire une image binaire indiquant quels pixels appartiennent à l'objet.

Dans un masque binaire, les pixels de l'objet valent généralement 1 et ceux du fond valent 0. Le masque conserve donc une approximation de la silhouette de l'objet. Contrairement à une simple *bounding box*, il permet de distinguer précisément le contour d'une assiette, d'une fourchette ou d'un clavier.

Les auteurs constatent que les boîtes et les masques de Mask R-CNN sont généralement exploitables, mais que les noms de classes peuvent être incorrects. Une assiette peut, par exemple, être reconnue comme un *frisbee*. Cette erreur serait particulièrement grave si le mot incorrect était transmis à DALL-E, car le modèle générerait alors une scène correspondant au mauvais objet.

3.2 Création automatique des légendes c_i avec OFA

Pour éviter de dépendre directement des classes prédites par Mask R-CNN, le système découpe l'image autour de la *bounding box* de chaque objet. Chaque morceau d'image, ou **object crop**, est ensuite fourni au modèle **OFA**.

OFA est ici utilisé comme modèle **image-to-text captioning** : il reçoit une image et produit automatiquement une phrase qui la décrit. La légende c_i n'est donc pas écrite manuellement par les chercheurs.

Exemples de légendes possibles :

- *"a fork with a black handle on a wooden table"* ;
- *"an empty white plate on a wooden table"* ;
- *"a red apple on a wooden table"*.

Cette étape apporte deux avantages :

1. le nom de l'objet peut être plus fiable que la classe brute de Mask R-CNN ;
2. la phrase peut contenir des propriétés visuelles supplémentaires, comme la couleur, la forme ou le matériau.

Dans la notation de l'article, c_i est donc **la légende textuelle automatiquement générée pour l'objet** i.

3.3 Extraction du vecteur visuel-sémantique v_i avec CLIP

Le même *crop* est aussi envoyé à la partie visuelle de **CLIP** (*Contrastive Language–Image Pre-training*).

CLIP transforme l'image de l'objet en un vecteur numérique de 512 dimensions :

$$v_i \in \mathbb{R}^{512}$$

Ce vecteur est appelé **visual-semantic feature vector**, ou vecteur de caractéristiques visuelles et sémantiques. Il encode non seulement des aspects visuels — forme, texture, couleur — mais aussi des informations liées au sens de l'objet. Deux images différentes représentant une assiette devraient ainsi produire des vecteurs plus proches que les vecteurs d'une assiette et d'un clavier.

Le vecteur ne correspond pas à une liste directement lisible de propriétés. Chaque composante est une valeur apprise par le réseau. C'est l'ensemble du vecteur, et surtout sa proximité avec d'autres vecteurs, qui est utile.

À la fin de cette phase, le système a transformé l'image initiale en une collection :

$$\{(M_i, c_i, v_i)\}_{i=1}^{n}$$

où n est le nombre d'objets détectés.

---

4. Construction automatique du *prompt* textuel

Les légendes complètes produites par OFA contiennent le nom des objets mais aussi des adjectifs et des informations de contexte. Dans cette expérience, les auteurs choisissent de n'utiliser que les noms des classes d'objets pour construire le *prompt*.

4.1 Extraction des noms

Les légendes c_i passent par un modèle de **Part-of-Speech tagging** de la bibliothèque Flair.

Le *Part-of-Speech tagging*, ou **étiquetage morpho-syntaxique**, consiste à attribuer une catégorie grammaticale à chaque mot :

- nom ;
- verbe ;
- adjectif ;
- préposition ;
- etc.

Par exemple, dans :

> *"a red apple on a wooden table"*

le système peut identifier *apple* comme le nom correspondant à la classe de l'objet. Les informations comme *red* ou *wooden* ne sont pas utilisées dans le *prompt* final de cette version de la méthode.

4.2 Assemblage du *prompt*

Les noms extraits sont réunis dans une phrase qui fait le moins d'hypothèses possible sur l'arrangement final. Pour une scène de table, le *prompt* peut être :

> *"A fork, a knife, a plate, and a spoon, top-down"*

L'expression **top-down** signifie « vu du dessus ». Elle encourage DALL-E à générer une scène avec un point de vue similaire à celui de la caméra du robot. Sans cette cohérence de perspective, la conversion des positions de l'image générée vers la table réelle serait beaucoup plus difficile.

Le *prompt* ne dit pas explicitement où placer chaque objet. Par exemple, il ne précise pas que la fourchette doit être à gauche de l'assiette. DALL-E doit exploiter les régularités apprises dans les images du Web pour proposer une organisation qu'un humain pourrait considérer comme naturelle.

---

5. Construction du masque d'image utilisé pour l'*inpainting*

DALL-E reçoit non seulement le texte ℓ, mais aussi un masque d'image noté I_M. Ce masque détermine les zones qui peuvent être modifiées et celles qui doivent rester fixes.

L'opération est une forme d'**inpainting** : le modèle complète ou remplace certaines régions d'une image tout en respectant les pixels conservés autour de ces régions.

Le masque répond à trois besoins.

5.1 Préserver les objets immobiles

Certains objets sont visibles mais ne doivent pas être déplacés. Par exemple :

- l'iPad dans la scène de bureau ;
- le panier dans la scène de fruits.

Leurs contours sont conservés dans le masque. DALL-E doit donc imaginer l'organisation des objets mobiles autour de ces éléments fixes. Cela permet aussi de placer un objet à l'intérieur d'un contenant immobile, comme un fruit dans un panier.

5.2 Conserver les bords de la table

Le système conserve également des zones autour des bords de la table. Cette opération est appelée **visual grounding**, ou ancrage visuel.

Elle donne à DALL-E une référence spatiale sur les limites de la scène. Elle réduit notamment le risque que le modèle génère des objets partiellement coupés au bord de l'image. Les auteurs observent aussi que cet ancrage encourage la génération d'objets à une échelle plus appropriée.

5.3 Effacer les objets mobiles et leurs ombres

Les masques des objets mobiles sont retirés de l'image afin que DALL-E puisse les replacer ailleurs. Ces régions sont légèrement agrandies pour supprimer également les ombres.

Cette précaution est importante : si l'ombre d'un objet reste visible à son emplacement initial, DALL-E peut interpréter cette ombre comme une contrainte et reconstruire l'objet au même endroit. Le système obtiendrait alors très peu de diversité dans les arrangements proposés.

---

6. Génération de plusieurs images cibles avec DALL-E 2

DALL-E 2 reçoit :

- le *prompt* textuel ℓ ;
- le masque d'image I_M ;
- les parties de l'image initiale qui doivent être conservées.

Le modèle produit une image candidate I_G, où G signifie *generated*. Mathématiquement, l'article représente cette génération par une distribution conditionnelle :

$$p_\theta(I_G \mid \ell, I_M)$$

Cette notation signifie que la probabilité de produire une certaine image dépend du texte et du masque fournis au modèle.

DALL-E est un **diffusion model**. De façon simplifiée, un modèle de diffusion apprend à transformer progressivement du bruit en image cohérente. Le texte guide ce débruitage vers les objets demandés et vers une composition compatible avec la description.

Les auteurs ne se contentent pas d'une seule génération. Ils demandent un **lot de quatre images**. Cette stratégie, appelée **sample-and-filter**, consiste à :

1. échantillonner plusieurs solutions possibles ;
2. éliminer les solutions inutilisables ;
3. conserver la meilleure solution selon un critère automatique.

Cette étape est nécessaire parce qu'un modèle génératif peut :

- ajouter un objet absent du *prompt* ;
- oublier un objet ;
- produire un artefact ;
- fusionner deux objets ;
- placer un objet partiellement hors de l'image ;

- générer une forme que le détecteur reconnaît mal.

Le système utilise DALL-E 2 tel qu'il existe déjà, à travers l'API d'OpenAI. Aucun réentraînement spécifique à ces expériences n'est effectué.

---

7. Analyse et filtrage des images générées

Chaque image générée est analysée avec la même logique que l'image réelle :

1. détection des objets ;
2. création de leurs masques ;
3. extraction de leurs vecteurs CLIP.

7.1 Filtrage par nombre d'objets

Le nombre d'objets détectés dans chaque image candidate est comparé au nombre d'objets de la scène initiale.

Une image est rejetée si les nombres ne correspondent pas. Par exemple, si la scène réelle contient une assiette, une fourchette, un couteau et une cuillère, une image qui contient cinq objets détectés ne peut pas être utilisée directement.

Si aucune des quatre images du lot ne contient le bon nombre d'objets, le système demande un nouveau lot à DALL-E.

Ce filtrage vérifie une condition de faisabilité fondamentale : le robot doit pouvoir établir une correspondance un-à-un entre les objets qu'il possède réellement et les objets représentés dans le but.

7.2 Calcul de la similarité CLIP

Pour comparer un objet réel et un objet généré, le système utilise la **cosine similarity**, ou similarité cosinus, entre leurs vecteurs CLIP.

Pour deux vecteurs v_a et v_b, elle est calculée par :

$$\mathrm{sim}(v_a, v_b) = \frac{v_a \cdot v_b}{\|v_a\| \, \|v_b\|}$$

Une valeur élevée indique que les deux représentations pointent dans des directions proches dans l'espace de caractéristiques et sont donc considérées comme sémantiquement similaires.

Cette comparaison doit fonctionner entre des **instances différentes**. L'assiette générée par DALL-E peut être carrée et l'assiette réelle ronde. Il ne faut donc pas comparer uniquement les pixels. CLIP sert à reconnaître qu'elles représentent malgré tout le même type d'objet.

---

8. Appariement des objets avec le *Hungarian Matching*

Supposons que la scène réelle contienne n objets et que l'image générée en contienne également n. Le système calcule la similarité CLIP entre chaque objet réel et chaque objet généré. Il obtient une matrice de scores.

Choisir, pour chaque objet, son meilleur partenaire indépendamment pourrait produire des conflits : deux objets réels pourraient être associés au même objet généré. Une sélection gloutonne, ou **greedy matching**, ne garantit pas non plus le meilleur score total.

Les auteurs utilisent donc l'**algorithme hongrois** (*Hungarian Matching algorithm*). Cet algorithme résout un problème d'affectation globale :

- chaque objet réel reçoit exactement un objet généré ;
- chaque objet généré est utilisé exactement une fois ;
- la somme totale des similarités est maximisée.

Pour chaque image candidate, le système obtient ainsi un score global d'appariement. Parmi les images qui ont le bon nombre d'objets, il conserve celle dont le score total est le plus élevé.

Cette image sélectionnée devient l'**image objectif** : elle décrit l'arrangement que le robot cherchera à reproduire.

---

9. Estimation de la pose cible de chaque objet

Après l'appariement, le système connaît pour chaque objet :

- son masque dans l'image réelle initiale ;
- le masque de l'objet correspondant dans l'image générée.

Il reste à calculer comment passer de la première pose à la seconde.

9.1 Mise à l'échelle des masques

Les objets générés par DALL-E n'ont pas forcément la même taille apparente que les objets réels. Le système redimensionne donc le masque initial afin que les dimensions de sa *bounding box* correspondent à celles du masque généré.

Cette mise à l'échelle ne signifie pas que le robot va agrandir physiquement l'objet. Elle sert uniquement à rendre les silhouettes comparables avant de calculer la translation et la rotation.

9.2 Alignement par ICP

Le système utilise ensuite **ICP**, pour *Iterative Closest Point*.

Chaque pixel appartenant au masque est considéré comme un point. ICP cherche la transformation qui aligne au mieux l'ensemble des points du masque réel avec ceux du masque généré. L'algorithme alterne généralement deux opérations :

1. trouver, pour les points d'un ensemble, leurs correspondants les plus proches dans l'autre ensemble ;
2. mettre à jour la transformation qui réduit les distances entre ces correspondances.

Le résultat est une transformation à trois degrés de liberté :

$$T_i = (x_i, y_i, \theta_i)$$

où :

- x_i est le déplacement horizontal dans l'image ;
- y_i est le déplacement vertical dans l'image ;
- θ_i est la rotation dans le plan de la table.

9.3 Correction de l'échelle globale et des collisions

Même après l'alignement, les distances entre objets peuvent être incorrectes parce que DALL-E peut générer les objets à une autre échelle. Une reproduction directe pourrait provoquer des collisions ou, au contraire, un espacement excessif.

Le système ajuste alors les positions par rapport à un objet central. Cet objet est choisi comme celui dont la somme des distances aux autres objets est minimale. Les objets sont rapprochés ou éloignés en fonction de la différence d'échelle observée.

Si des masques se chevauchent encore dans l'arrangement calculé, les objets sont progressivement éloignés de l'objet central jusqu'à supprimer les collisions.

Cette correction transforme donc une composition visuellement plausible en une disposition plus facilement réalisable dans le monde physique.

---

10. Passage des pixels aux coordonnées réelles

Les poses calculées jusque-là sont exprimées dans le repère 2D de l'image. Le robot, lui, a besoin de coordonnées dans son espace de travail.

La caméra montée sur le poignet fournit également une information de profondeur. Cette profondeur permet de projeter les positions des pixels dans l'espace 3D et de retrouver leur emplacement sur le plan de la table.

Le système convertit ainsi chaque transformation (x, y, θ) de l'image en une transformation cartésienne utilisable par le robot.

Cette opération nécessite implicitement une relation géométrique connue entre :

- la caméra ;
- le poignet du robot ;
- le repère du robot ;
- le plan de la table.

Le résultat final de cette étape est, pour chaque objet réel, une pose cible physiquement définie.

---

11. Exécution avec le robot

Les expériences physiques utilisent un robot **Franka Emika Panda** équipé d'une caméra au poignet et d'une ventouse souple, appelée **compliant suction gripper**.

Le robot exécute une suite d'opérations **pick-and-place** :

1. se déplacer au-dessus de l'objet ;
2. descendre vers l'objet ;
3. saisir l'objet avec la ventouse ;

4. le soulever ;
5. se déplacer vers la pose cible ;
6. appliquer la rotation voulue ;
7. déposer l'objet ;
8. relâcher la ventouse.

L'ordre des déplacements est important. Un objet peut occuper momentanément la future position d'un autre. Les auteurs ajoutent donc un planificateur simple : lorsqu'un mouvement direct provoquerait un conflit, le robot place d'abord certains objets dans des positions intermédiaires situées sur le côté, puis les déplace vers leurs positions finales.

DALL-E ne produit donc jamais directement une commande moteur. La chaîne est :

$$\text{image imaginée} \rightarrow \text{objets appariés} \rightarrow \text{poses cibles} \rightarrow \text{trajectoires pick-and-place}$$

---

12. Expérience 1 : réarrangement autonome *zero-shot*

12.1 Question expérimentale

La première expérience cherche à répondre à la question :

> DALL-E-Bot peut-il organiser un ensemble d'objets d'une manière que les humains préfèrent ?

12.2 Scènes testées

Trois scènes de la vie quotidienne sont utilisées.

Scène de repas (*dining scene*)

Elle contient quatre objets mobiles :

- une assiette ;
- une fourchette ;
- un couteau ;
- une cuillère.

Cette scène teste fortement la connaissance sémantique, car les couverts et l'assiette ont des relations spatiales culturellement fréquentes.

Scène de bureau (*office scene*)

Elle contient :

- un clavier ;
- une souris ;
- une tasse ;
- un iPad immobile.

L'iPad est visible, mais le robot n'a pas le droit de le déplacer. Son emplacement doit donc être préservé par le masque d'*inpainting*.

Scène de fruits (*fruit basket scene*)

Elle contient :

- deux pommes ;
- une orange ;
- un panier immobile.

Le système doit comprendre qu'une disposition naturelle peut consister à placer les fruits dans le panier, même si cette règle n'est pas explicitement écrite dans le *prompt*.

12.3 Méthodes de comparaison

Les auteurs comparent DALL-E-Bot à deux méthodes heuristiques *zero-shot*.

Rand-No-Coll place les objets au hasard, tout en empêchant leurs chevauchements. Ce *baseline* indique ce que donnerait une organisation sans raisonnement sémantique.

Geometric place les objets régulièrement sur une ligne droite, sans collision, et utilise leurs *bounding boxes* pour les rendre parallèles. Cette méthode produit une structure visuelle ordonnée, mais ne connaît pas la fonction des objets.

Deux **ablation studies** sont également réalisées. Une ablation retire ou remplace un composant afin de mesurer son importance.

Abl-Mask-RCNN utilise directement les classes prédites par Mask R-CNN pour le *prompt*, à la place des légendes produites par OFA. Cette variante teste l'intérêt du modèle de *captioning*.

Abl-No-VG retire l'ancrage visuel des bords de la table. Cette variante teste l'utilité du *visual grounding*.

12.4 Réalisation physique avant évaluation

Le jugement ne porte pas uniquement sur les images produites par DALL-E. Le robot réalise réellement chaque arrangement, puis une nouvelle photographie est prise.

Cette décision est importante, car elle inclut toutes les erreurs possibles de la chaîne :

- mauvaise détection ;
- mauvais appariement ;
- erreur d'alignement des masques ;
- erreur de calibration ;
- glissement lors de la saisie ;
- imprécision du dépôt.

L'évaluation mesure donc le résultat pratique du système complet et non une version idéale limitée au monde numérique.

12.5 Étude utilisateur

Les arrangements sont évalués par 17 participants âgés de 20 à 71 ans. Pour chacune des trois scènes et chacune des cinq méthodes, cinq arrangements provenant d'initialisations différentes sont présentés. L'étude rassemble 1 275 notes.

La question posée est :

> *"If the robot made this arrangement for you, how happy would you be?"*

Autrement dit : « Si le robot réalisait cet arrangement pour vous, à quel point seriez-vous satisfait ? »

Les participants répondent sur une **échelle de Likert** allant de 1 à 10 :

- 1 signifie *very unhappy* ;
- 10 signifie *very happy*.

Tous les participants voient les mêmes images, ce qui rend la comparaison entre méthodes plus cohérente.

12.6 Résultats de l'étude utilisateur

Les moyennes rapportées dans la version finale de l'article sont :

Méthode	Repas	Bureau	Fruits	Moyenne
Rand-No-Coll	2,21	3,42	3,26	2,96
Geometric	4,14	3,47	2,85	3,49
Abl-Mask-RCNN	3,99	7,53	5,86	5,79
Abl-No-VG	7,45	7,13	5,48	6,69
DALL-E-Bot	7,14	7,71	9,55	8,13

DALL-E-Bot obtient la meilleure moyenne globale. Les résultats soutiennent plusieurs conclusions :

- une disposition géométriquement propre n'est pas suffisante ;
- les utilisateurs valorisent les relations sémantiques entre les objets ;
- les mauvaises classes de Mask R-CNN peuvent fortement détériorer le *prompt* ;
- le modèle de *captioning* OFA est donc utile ;
- l'ancrage visuel aide à produire des images plus exploitables ;
- l'utilisation d'un modèle génératif pré-entraîné permet d'obtenir des arrangements jugés naturels sans entraînement propre à ces trois scènes.

Il faut toutefois interpréter ces résultats correctement : l'expérience démontre une performance encourageante sur trois types de scènes, mais ne prouve pas que le système fonctionnerait avec n'importe quelle scène ou n'importe quel objet.

---

13. Expérience 2 : placement d'un objet manquant par *inpainting*

13.1 Question expérimentale

La seconde expérience cherche à mesurer plus objectivement la précision du système :

> DALL-E-Bot peut-il compléter correctement une disposition partiellement créée par un humain ?

Contrairement à l'expérience précédente, il ne faut pas réorganiser toute la scène. Les objets déjà placés doivent rester fixes et un seul objet manquant doit être inséré.

13.2 Création des arrangements de référence

Dix participants, droitiers et gauchers, réalisent leur propre disposition de table à partir de l'assiette, de la fourchette, du couteau et de la cuillère.

La consigne leur demande d'imaginer qu'ils vont s'asseoir pour dîner et d'organiser les objets de façon satisfaisante. Pour chaque objet, les participants peuvent aussi indiquer plusieurs poses alternatives qu'ils jugent acceptables, les autres objets restant fixes.

Cette possibilité est essentielle, car il n'existe pas toujours une unique bonne réponse. Une fourchette peut être placée à plusieurs distances légèrement différentes tout en restant acceptable. La distribution des solutions humaines est donc **multimodale** : elle contient plusieurs modes, ou plusieurs groupes de solutions valides.

13.3 Retrait virtuel d'un objet

Pour chaque arrangement humain, le système choisit successivement un objet comme objet manquant.

Les autres objets sont protégés dans le masque d'*inpainting*. DALL-E ne peut pas modifier leurs pixels. La zone restante est ouverte à la génération, et le *prompt* demande l'ensemble des objets de la scène.

DALL-E doit alors générer l'objet absent à une position cohérente par rapport aux objets déjà fixés.

13.4 Mesure des erreurs

Le masque de l'objet dans la pose prédite est aligné avec le masque du même objet dans la pose choisie par l'humain. Ici, il s'agit de la même instance physique d'objet, et non de deux objets simplement similaires. L'alignement est donc plus précis que dans le cas général entre objet réel et objet généré.

Deux erreurs sont calculées :

- l'**erreur de position**, mesurée en centimètres ;
- l'**erreur d'orientation**, mesurée en degrés.

La prédiction est comparée à toutes les poses déclarées acceptables par le participant, puis l'erreur par rapport à la pose acceptable la plus proche est retenue.

Les auteurs rapportent la **médiane** plutôt que la moyenne. La médiane est moins influencée par quelques grandes erreurs et représente mieux le comportement typique lorsque la distribution contient des valeurs extrêmes.

L'orientation de l'assiette n'est pas rapportée, car une assiette ronde n'a pas d'orientation visuelle pertinente dans le plan.

13.5 *Baselines* de la seconde expérience

Rand-No-Coll place l'objet manquant au hasard dans la zone autorisée, sans collision avec les objets fixes.

Geometric construit une ligne à partir des centroïdes des masques de deux objets fixes. L'objet manquant est placé sur cette ligne, aussi près que possible des autres sans collision, et orienté parallèlement à l'objet fixe le plus proche.

13.6 Résultats quantitatifs

Les valeurs suivantes sont les erreurs médianes de position et d'orientation :

Méthode	Fourchette	Assiette	Cuillère	Couteau
Rand-No-Coll	25,85 cm / 70,32°	10,78 cm / —	27,47 cm / 42,56°	23,51 cm / 99,32°
Geometric	15,59 cm / 40,57°	2,29 cm / —	23,83 cm / 86,11°	11,58 cm / 1,47°
DALL-E-Bot	4,95 cm / 1,26°	1,28 cm / —	2,13 cm / 2,72°	2,10 cm / 3,27°

DALL-E-Bot réduit fortement les erreurs pour la majorité des objets. Cela indique que l'*inpainting* ne place pas seulement un objet dans une zone libre : il tient compte de la configuration des objets déjà présents.

Cette expérience illustre aussi une forme de collaboration humain–robot. Un humain peut commencer un arrangement, puis le robot peut tenter de le compléter sans modifier les éléments déjà placés.

---

14. Pourquoi l'approche est considérée comme *zero-shot*

Le terme *zero-shot* ne signifie pas que les modèles n'ont jamais été entraînés. DALL-E, CLIP, OFA et Mask R-CNN ont tous été pré-entraînés sur de grandes quantités de données.

Il signifie que, pour les tâches précises évaluées dans l'article :

- les auteurs ne collectent pas un nouveau jeu de données d'arrangements ;
- ils ne montrent pas au système des exemples de la disposition cible avant le test ;
- ils ne réentraînent pas DALL-E pour le dressage de table, le bureau ou les fruits ;
- ils ne font pas de *fine-tuning* propre à ces scènes ;
- les modèles sont utilisés directement comme composants déjà pré-entraînés.

La connaissance permettant de placer une fourchette près d'une assiette ou des fruits dans un panier provient indirectement des régularités présentes dans les très nombreuses paires image–texte utilisées lors du pré-entraînement de DALL-E.

Il est donc plus exact de dire : **zero-shot par rapport aux nouvelles tâches de réarrangement**, et non « sans aucun entraînement préalable ».

---

15. Rôle précis de chaque technique principale

Technique	Entrée	Sortie	Rôle dans DALL-E-Bot
Mask R-CNN	Image RGB	Boîtes, masques, classes	Localiser et segmenter les objets
OFA *image captioning*	*Crop* d'un objet	Légende c_i	Nommer automatiquement l'objet
Flair POS tagging	Légende c_i	Catégories grammaticales	Extraire le nom de la classe pour le *prompt*

Technique	Entrée	Sortie	Rôle dans DALL-E-Bot
CLIP	*Crop* d'un objet	Vecteur v_i de 512 dimensions	Comparer sémantiquement objets réels et générés
DALL-E 2	Texte, image et masque	Images candidates	Imaginer des arrangements naturels
Sample-and-filter	Plusieurs images candidates	Une image retenue	Écarter les générations non réalisables
Hungarian Matching	Matrice de similarités	Affectation globale des objets	Associer chaque objet réel à un objet généré
ICP	Deux masques correspondants	Translation et rotation	Estimer la pose cible dans l'image
Caméra de profondeur	Pixels et profondeur	Coordonnées 3D	Convertir la pose image en pose robotique
Pick-and-place	Poses initiales et cibles	Mouvements physiques	Réaliser l'arrangement sur la table

16. Limites techniques importantes

16.1 Erreurs de perception cumulées

Le système est une chaîne de plusieurs modèles. Une erreur au début peut se propager : une mauvaise segmentation entraîne un mauvais *crop*, qui peut produire une mauvaise légende, puis un mauvais *prompt* et finalement une mauvaise image cible.

16.2 Objets symétriques

L'alignement de masques est ambigu pour des objets symétriques. Un couteau ou un clavier peut parfois sembler correctement aligné après une rotation de 180°, alors que son sens fonctionnel est inversé.

Le masque binaire décrit la forme mais pas nécessairement la différence sémantique entre le manche et la lame. Les auteurs proposent comme amélioration possible d'aligner des cartes de caractéristiques sémantiques plutôt que de simples silhouettes binaires.

16.3 Différence entre objet réel et objet généré

DALL-E ne reproduit pas exactement l'instance réelle. Il peut générer une assiette d'une autre forme, un fruit d'une autre taille ou un clavier d'un autre style. CLIP aide à reconnaître les catégories, mais l'estimation géométrique devient plus difficile lorsque les formes diffèrent fortement.

16.4 Comptage et artefacts de génération

Les modèles texte-image peuvent produire le mauvais nombre d'objets, des objets fusionnés ou des objets partiellement visibles. Le filtrage limite le problème sans le supprimer. Il peut aussi être nécessaire de générer plusieurs lots avant d'obtenir une image utilisable.

16.5 Relations spatiales exprimées en langage

Le système pourrait théoriquement recevoir des demandes comme *"plates stacked"* ou *"the cup to the left of the keyboard"*. L'article souligne cependant que les modèles texte-image de cette génération ne respectent pas toujours précisément les relations spatiales formulées en langage.

16.6 Validité limitée des expériences

Les expériences utilisent trois familles de scènes sur une table et un ensemble restreint d'objets. Elles montrent la faisabilité de l'approche, mais ne démontrent pas une robustesse universelle dans des environnements encombrés, en 3D complexe ou avec des objets difficiles à saisir.

16.7 Génération plausible n'est pas compréhension parfaite

Une image peut paraître naturelle sans correspondre à une configuration physiquement sûre ou réalisable. Le système doit donc ajouter des vérifications de collisions, des corrections d'échelle et un planificateur robotique. Le modèle génératif apporte un **prior sémantique et esthétique**, mais il ne remplace pas les contraintes géométriques et physiques.

---

17. Interprétation générale

L'apport principal de l'article n'est pas que DALL-E sache contrôler seul un robot. L'apport est la transformation d'une connaissance visuelle apprise à l'échelle du Web en un objectif robotique concret.

DALL-E fournit une hypothèse sur **ce à quoi la scène devrait ressembler**. Les autres composants répondent ensuite aux questions nécessaires à l'action :

- Quels objets sont réellement présents ?
- Quel objet généré correspond à quel objet réel ?
- Où se trouve sa position cible ?
- Quelle rotation faut-il appliquer ?
- Comment convertir cette pose en coordonnées physiques ?
- Dans quel ordre déplacer les objets sans collision ?

La contribution est donc une architecture modulaire reliant trois niveaux :

1. **niveau sémantique** : comprendre quels objets sont présents et imaginer une organisation plausible ;
2. **niveau géométrique** : transformer cette organisation en positions et orientations ;
3. **niveau robotique** : exécuter les déplacements dans le monde réel.

Les résultats suggèrent qu'un modèle de diffusion pré-entraîné peut servir de **prior sur les préférences humaines** : il fournit une tendance générale sur la manière dont les objets sont habituellement organisés, sans que les chercheurs aient à encoder manuellement toutes les règles.

---

18. Référence principale

Ivan Kapelyukh, Vitalis Vosylius et Edward Johns, *DALL-E-Bot: Introducing Web-Scale Diffusion Models to Robotics*, IEEE Robotics and Automation Letters, 2023.

- Article : https://arxiv.org/abs/2210.02438
- Page du projet et vidéos : https://www.robot-learning.uk/dall-e-bot

Ce document suit la version révisée v3 de l’article disponible sur arXiv. Les chiffres expérimentaux peuvent différer de versions préliminaires du manuscrit.